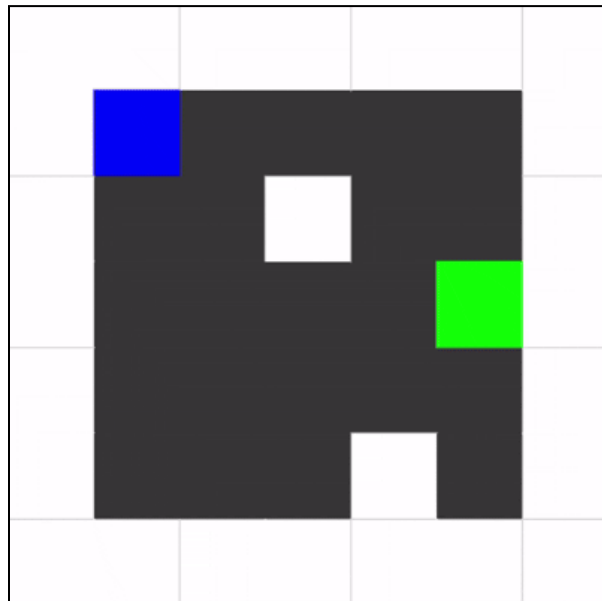


Tugas Kecil 3 IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Ice Sliding Puzzle Solver



Oleh :

Muhammad Nafis Habibi 13524018

Dzaki Ahmad Al Hussainy 13524084

Program Studi Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung 2025/2026

Daftar Isi

Daftar Isi.....	2
1. Deskripsi Tugas.....	3
1.1. Latar Belakang.....	3
1.2. Tugas.....	3
1.3. Batasan Masalah.....	3
2. Penyelesaian Permasalahan.....	4
2.1. Dekomposisi dan Implementasi Game.....	4
2.2. Representasi Tile.....	4
2.3. Representasi Player.....	6
2.4. Representasi Pergerakan Player.....	6
2.5. Representasi Status game.....	7
2.6. Representasi Board.....	7
2.7. Algoritma Pathfinding.....	9
3. Arsitektur Program.....	21
3.1. Struktur Perangkat Lunak.....	21
3.2. Bonus GUI.....	21
4. Pengujian.....	23
5. Kesimpulan dan Saran.....	30
Lampiran.....	31
Laporan Pengerjaan.....	32
Daftar Pustaka.....	33

1. Deskripsi Tugas

1.1. Latar Belakang

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih).

1.2. Tugas

Buatlah sebuah program tersebut dalam bahasa Java, Python, C++, C, atau GO yang mengimplementasikan algoritma Pathfinding (UCS, GBFS, dan A*) untuk mencari solusi dalam permainan Ice Sliding Puzzle Solver.

1.3. Batasan Masalah

1.3.1. Aturan Permainan

- **Pergerakan:** Pemain bergerak dalam satu arah (atas, kiri, bawah, kanan) sampai menabrak dinding.
- **Angka:** Ambil angka dengan melewati petak angka. Pengambilan harus secara berurutan ($0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n$). Angka yang lebih kecil harus diambil terlebih dahulu jika ingin mengambil angka yang lebih besar.
- **Kekalahan:** Jika pemain melewati petak lava saat bergerak, pemain akan kalah. Jika bergerak keluar dari papan, pemain juga kalah. Jika pemain mengambil angka tidak sesuai urutan, pemain juga kalah.
- **Kemenangan:** Pemain menang jika berhasil berhenti di titik finish (petak 'O'). Titik finish tidak otomatis menghentikan pergerakan. Pemain dapat melewatinya begitu saja jika tidak ada yang menghentikan pemain. Semua angka tidak harus diambil untuk menang.

1.3.2. Input:

- File .txt berisi konfigurasi dari map yang diberikan.
 - Baris pertama memuat 2 angka (N, M) yang menyatakan panjang dan lebar papan
 - N baris berikutnya merepresentasikan papan
 - N baris berikutnya merepresentasikan cost untuk melewati tile tersebut
- Detail representasi tile dapat dilihat di bagian 2.2.

1.3.3. Output:

- Menampilkan solusi gerakan yang ditemukan

- Menampilkan cost dari solusi
- Menampilkan visualisasi dari papan yang di mana oleh pemain bisa mencapai titik tujuan dengan algoritma pathfinding yang digunakan
- Menampilkan banyak konfigurasi atau iterasi yang ditinjau oleh algoritma, yang nantinya tiap iterasinya akan dapat disimpan dalam .txt
- Menampilkan waktu eksekusi program dalam milisecond (cukup waktu eksekusi, tidak termasuk membaca dan menulis file).
- Playback (bukan Live Update): program harus dapat memvisualisasikan proses pathfinding yang dilakukan. Visualisasi proses dilakukan setelah proses pencarian selesai dilakukan. Jika membuat bonus GUI, pastikan playback dapat dijalankan dengan kecepatan yang dapat diatur selayaknya video, serta dapat bergerak maju maupun mundur.

2. Penyelesaian Permasalahan

2.1. Dekomposisi dan Implementasi Game

Dalam dekomposisi game ini adalah bagaimana kami merepresentasikan elemen-elemen dalam game ke dalam program. Kami menggunakan bahasa Java untuk melakukan implementasi dari permainan ini. Pada penjelasan representasi masing-masing dekomposisi kami tidak menuliskan semua getter setter pada masing-masing kelas yang dipakai. Laporan ini hanya berisi representasi penting dari logika program yang dipakai. Namun, tentunya kami menggunakan penamaan method yang menggambarkan getter dan setter yang tidak tertulis untuk bagian logika nanti sehingga pembaca bisa memahami logika program dengan baik.

2.2. Representasi Tile

Pada permainan ini dijelaskan petak atau tile masing masing memiliki harga lewat dan dapat berbentuk dari jenis berikut:

- 1) * = path yang bisa dilewati
- 2) X = Rintangan/Batu. Aktor akan berhenti tepat sebelum batu.
- 3) L = Lava. Melewati lava akan mengalami game over (Meskipun tidak berhenti tepat di lava).
- 4) Z = Aktor/Pengguna
- 5) O = Titik tujuan
- 6) <i> = Angka di sini kami menganggap angka adalah coin karena lebih keren aja.

Aturan untuk angka ini Aktor harus melewati angka ini sesuai dengan urutan angka di papan. Petak dengan angka 1 harus dilewati setelah aktor melewati petak dengan angka 0, Petak dengan angka 2 harus dilewati setelah aktor melewati petak dengan angka 1, dan seterusnya. (Maksimal hanya sampai angka 9). Apabila aturan ini dilanggar, misalnya petak 1 diinjak sebelum petak 0, maka akan game-over.

Untuk menangani ini kami melakukan abstraksi dan membuat enumerasi bernama TileType dan kelas Tile untuk merepresentasikan tile ini.

```
public enum TileType {
    PATH('*', true),
    OBSTACLE('X', false),
    LAVA('L', true),
    GOAL('O', true),
    COIN_NUMBER('i', true),
    COIN_COLLECTED('i', true),
    PLAYER('Z', true);

    private final char symbol;
    private final boolean traversable;
}
```

Enumerasi TileType ini digunakan untuk mendapatkan jenis dari tile dan mengecek apakah tile tersebut bisa dilewatkan atau tidak.

Sehingga dapat dibuat kelas tile seperti ini yang memiliki lokasi koordinat x, y, tile petak, harga traversal serta urutan coin (angka). Di dalam spesifikasi di detailkan akan ada urutan angka yang harus diambil sesuai urutan.

```
public class Tile {
    private int x;
    private int y;

    private TileType type;
    private int cost;
    private int coinSequence;

    public Tile(int x, int y, TileType type, int cost) {
        this.x = x;
        this.y = y;
        this.type = type;
    }
}
```

```

        this.cost = cost;
    }

    public void setCoinSequence(int value){
        if (value < 1 && value > 9) {
            throw new IllegalArgumentException("Input angka " + value + " diluar batas
1-9");
        }
        if (type != TileType.COIN_NUMBER) {
            throw new RuntimeException("Gagal set angka! Tipe tile saat ini adalah " +
this.type.name());
        }

        this.coinSequence = value;
    }
}

```

Di dalam kelas tile ini juga dipakai pengecekan apakah tile adalah sebuah lava, coin, atau obstacle untuk membuat enkapsulasi kelas tile yang bisa dipakai secara modular.

2.3. Representasi Player

Berikut adalah abstraksi dari sebuah pemain pemain akan memiliki koordinat lokasi di dalam sebuah board, memiliki status apakah mati dan total hasil traversal dari pemain.

```

public class Player {
    int x;
    int y;
    boolean isAlive;
    long totalCost;
}

```

2.4. Representasi Pergerakan Player

Abstraksi dalam pergerakan pemain digambarkan dengan penggunaan enumerasi Direction yang memiliki pergerakan arah x dan y. Masing masing enumerasi memiliki pergerakan ke atas, bawah, kiri dan kanan seperti implementasi dibawah ini.

```

public enum Direction {
    UP(0, -1),

```

```

    DOWN(0, 1),
    LEFT(-1, 0),
    RIGHT(1, 0);

    public final int dx;
    public final int dy;
}

```

2.5. Representasi Status game

Untuk menggambarkan keadaan permainan digunakan enumerasi bernama

```

public enum GameStatus {
    PLAYING,
    WON,
    GAME_OVER
}

```

2.6. Representasi Board

Dalam menggambarkan keadaan board saat ini dibuatkan sebuah kelas board seperti ini.

```

public class Board {
    private int columnSize;
    private int rowSize;
    private Player player;
    private GameStatus gameStatus;
    private Queue<Integer> coinOrder = new LinkedList<>();
    private Map<Integer, Tile> coinMap = new HashMap<>();
    private Tile goalTile;

    private int initialPlayerPosX;
    private int initialPlayerPosY;

    private List<Tile> tiles;

    public Tile getNextTarget() {
        if (!coinOrder.isEmpty()) {
            return coinMap.get(coinOrder.peek());
        }
    }
}

```

```

    }
    return goalTile;
}
}

```

Kelas Board dirancang untuk merepresentasikan keadaan (state) permainan secara menyeluruh, mencakup dimensi papan, entitas di dalamnya (pemain dan koin), serta logika menggerakkan player dan tujuan berikutnya.

Di sini menggunakan struktur data queue pada coinOrder yang digunakan untuk menyimpan urutan ID koin yang harus diambil oleh pemain. Penggunaan antrean (Queue) memastikan pemain mengambil koin sesuai urutan tertentu (FIFO). Digunakan juga Map yang memetakan ID koin ke lokasi fisik tile pada papan untuk pencarian yang efisien.

Metode getNextTarget fungsinya adalah menentukan koordinat atau petak mana yang harus dituju oleh pemain selanjutnya yang akan mengutamakan coin kemudian goal.

Terakhir, board ini memiliki logika pergerakan player untuk menggerakkan player pada arah tertentu pada board permainan dengan mengikuti aturan dari permainan ini.

```

public void movePlayer(Direction playerDirection) throws GameOverException {
    while (gameStatus == GameStatus.PLAYING) {
        int nextX = player.x + playerDirection.dx;
        int nextY = player.y + playerDirection.dy;
        if (isOutOfBound(nextX, nextY)) {
            if (player.x == goalTile.getXCoords() && player.y == goalTile.getYCoords()) {
                gameStatus = GameStatus.WON;
                System.out.println("WIN : PLAYER BERHASIL KE GOAL");
            } else {
                gameStatus = GameStatus.GAME_OVER;
                System.out.println("GAME OVER : PLAYER KELUAR DARI PAPAN");
            }
            break;
        }
        Tile targetTile = getTileAt(nextX, nextY);
        if (targetTile.isObstacle()) {
            if (player.x == goalTile.getXCoords() && player.y == goalTile.getYCoords()) {
                gameStatus = GameStatus.WON;
                System.out.println("WIN : PLAYER BERHASIL KE GOAL");
            }
            break;
        }
    }

    player.x = nextX;
}

```



```

player.y = nextY;
player.setTotalCost(player.getTotalCost() + targetTile.getTileCost());

if (targetTile.isLava()) {
    player.setPlayerStatus(false);
    gameStatus = GameStatus.GAME_OVER;
    System.out.println("GAME OVER : PLAYER TERKENA LAVA");
    break;
}

if (targetTile.isCoin()) {
    int currentCoinSequence = targetTile.getCoinSequence();
    if (coinOrder.isEmpty()) {
        break;
    }
    if (coinOrder.peek() == currentCoinSequence) {
        coinOrder.poll();
        targetTile.setType(TileType.COIN_COLLECTED);
    } else if (currentCoinSequence > coinOrder.peek()) {
        this.gameStatus = GameStatus.GAME_OVER;
        throw new GameOverException("Urutan salah! Kamu menginjak " + currentCoinSequence
            + " padahal seharusnya " + coinOrder.peek());
    }
}
}
}

```

2.7. Algoritma Pathfinding

2.7.1. Tree Search

Dalam menyelesaikan permasalahan pencarian ini diperlukan permulaan dari gerak player yang bisa memilih ke UP,DOWN,LEFT,RIGHT. Dengan lokasi pertama pada lokasi awal player. Kemudian jika pergerakannya valid akan dapat membuat search tree dari posisi awal ke posisi yang berikutnya berdasarkan aturan dan syarat kemenangan yang telah ditentukan.

Sehingga diperlukan sebuah node yang akan mencatat keadaan masing-masing state dari pencarian untuk mendapatkan solusi optimal.

```

public class Node {
    private final int x, y;
    private final Queue<Integer> remainingCoins;
    private long gCost;
}

```

```
private long hCost;
private final Node parent;
private final Direction lastMove;
}
```

Abstraksi dari bode ini akan menggambarkan state dari sebuah pencarian mulai dari lokasi pemain, urutan koin yang diambil, nilai harga sebenarnya, harga heuristik dan node orang tua dan gerakan terakhir untuk mencapai node itu.

Dalam pencarian path finding ini diperlukan sebuah algoritma untuk mensimulasikan dan melakukan searching namun tidak langsung pada board pemain sehingga dibuat implementasi pergerakan pada node seperti ini

```
public Node move(Direction dir, Board board) {
    int curX = this.x;
    int curY = this.y;
    long moveCost = 0;
    Queue<Integer> newQueue = new LinkedList<>(this.remainingCoins);

    while (true) {
        int nextX = curX + dir.getDx();
        int nextY = curY + dir.getDy();

        // Pemain akan mati ketika melewati batas papan
        if (board.isOutOfBound(nextX, nextY)) {
            return null;
        }

        // Pemain akan berhenti ketika terkena obstacle
        // (Rintangan/batu)
        if (board.getTileAt(nextX, nextY).isObstacle()) {
            break;
        }

        curX = nextX;
        curY = nextY;
        Tile target = board.getTileAt(curX, curY);
        moveCost += target.getTileCost();
    }
}
```

```

        // Pemain akan mati ketika terkena dengan lava
        if (target.isLava()) return null;

        // Melakukan cek terhadap urutan coin atau checkpoint
        if (target.isCoin()) {
            int val = target.getCoinSequence();

            // jika koin sudah habis atau koin yang diambil sesuai
urutan
            if (!newQueue.isEmpty() && val == newQueue.peek()) {
                newQueue.poll();
                // ketikacheckpoin koin yang diambil salah urutan maka
pemain kalah
            } else if (!newQueue.isEmpty() && val > newQueue.peek())
        {
            return null;
        }
    }

    // jika sampai di goal dan sudah tidak ada urutan koin atau
checkpoint maka menang
    if (target.isGoal() && newQueue.isEmpty()) break;
}

// pemain jika berada di tempat yang sama (seharusnya tidak
sampai disini)
if (curX == this.x && curY == this.y) return null;

// mengembalikan node dengan posisi baru, urutan koin baru, hasil
pergerakan baru, tanpa heuristik, parent node ini, pergerakan
sebelumnya
return new Node(curX, curY, newQueue, this.gCost + moveCost, 0,
this, dir);
}

```

2.7.2. Implementasi Kontrak Penyelesaian Algoritma

Karena kami menggunakan Java pada penyelesaian ini, kami menggunakan pendekatan pemrograman berbasis objek dalam menulis solusi algoritma ini. Kami membuat kelas abstrak Solver yang berisi atribut board, dan

sebuah kontrak solve yang akan menghasilkan urutan arah apa saja untuk mendapatkan hasil cost yang minimum.

```
public abstract class Solver {
    protected Board board;
    public Solver(Board board) {
        this.board = board;
    }
    public abstract List<Direction> solve();
}
```

Pada kelas solver ini juga diimplementasikan sebuah fungsi reconstruct path yang berfungsi untuk mendapatkan arah optimal setelah solver menemukan solusi dan mencapai titik akhir.

```
public List<Direction> reconstructPath(Node node) {
    List<Direction> path = new ArrayList<>();
    Node curr = node;
    while (curr != null && curr.getLastMove() != null) {
        path.add(curr.getLastMove());
        curr = curr.getParent();
    }
    Collections.reverse(path);
    return path;
}
```

Kemudian untuk implementasi algoritma GBFS dan A* kami menggunakan heuristik yang sama yaitu dengan menggunakan perhitungan jarak Manhattan yaitu perhitungan jarak dari selisih absolut koordinat kartesiusnya. Alasan kami menggunakan jarak Manhattan ini juga karena jarak manhattan ini sudah terbukti *admissible* yaitu apabila nilai estimasinya tidak pernah melebihi biaya aktual minimum untuk mencapai tujuan dari simpul n. Secara formal: $h(n) \leq h^*(n)$, untuk setiap simpul n. Dengan $h^*(n)$ adalah biaya optimal sesungguhnya dari n ke tujuan.

1. Pada *Ice Sliding Puzzle*, pemain bergerak secara bebas ke empat arah (atas, bawah, kiri, kanan) hingga menabrak dinding. Jarak yang sesungguhnya diperlukan selalu lebih besar atau sama dengan jarak Manhattan, karena pemain tidak dapat berhenti di sembarang sel.

2. Karena $h(n)$ tidak pernah melebihi biaya aktual, heuristik ini memenuhi syarat *admissibility*.

Heuristik jarak Manhattan ini dipakai untuk menghitung posisi pemain dengan jarak target berikutnya. Target berikutnya bisa berupa coin atau berupa goal akhir.

```
public long calculateHeuristic(int curX, int curY, Queue<Integer>
currentCoins, Board board) {
    int targetX, targetY;

    if (!currentCoins.isEmpty()) {
        // jika target adalah koordinat koin berikutnya
        int nextCoin = currentCoins.peek();
        Tile coinTile = board.findCoinTile(nextCoin);
        if (coinTile == null) {
            throw new IllegalStateException(
                "Error: Koin " + nextCoin + " terdaftar di Queue
tapi tidak ditemukan di Board Map!");
        }
        targetX = coinTile.getXCoords();
        targetY = coinTile.getYCoords();
    } else {
        // jika target adalah Goal
        targetX = board.getGoalTile().getXCoords();
        targetY = board.getGoalTile().getYCoords();
    }

    // Manhattan Distance
    return Math.abs(curX - targetX) + Math.abs(curY - targetY);
}
```

Pada setiap implementasi path search ini akan menggunakan coin/number/check poin dan goal sebagai tujuan utama.

2.7.3. Greedy Best First Search (GBFS)

Greedy Best-First Search (GBFS) merupakan algoritma pencarian yang mengekskansi simpul berdasarkan estimasi jarak terdekat menuju target dengan menggunakan fungsi heuristik $h(n)$. GBFS mengabaikan biaya aktual yang telah ditempuh dari titik awal $g(n)$ dan sepenuhnya berfokus pada efisiensi pencarian menuju tujuan.

Dalam implementasi ini digunakan priority queue yang mengutamakan HCost terkecil ke yang terbesar. Agar tidak terjadi penelusuran ke node yang sama dipakai pula data struktur Set yang bernama visited. Set ini akan berisi kode keadaan dalam bentuk string dengan format “x, y, sisakoin”.

```
public class GBFSSolver extends Solver {

    public GBFSSolver(Board board) {
        super(board);
    }

    @Override
    public List<Direction> solve() {
        PriorityQueue<Node> pq = new
        PriorityQueue<>(Comparator.comparingLong(Node::getHCost));

        Set<String> visited = new HashSet<>();

        int startX = board.getPlayer().getXCoords();
        int startY = board.getPlayer().getYCoords();

        Queue<Integer> initialCoins = new
        LinkedList<>(board.getCoinOrder());

        long hInitial = calculateHeuristic(startX, startY, initialCoins,
        board);

        Node StartNode = new Node(startX, startY, initialCoins, 0,
        hInitial, null, null);
        pq.add(StartNode);
        while (!pq.isEmpty()) {
            Node current = pq.poll();
            String stateId = current.getX() + "," + current.getY() + ","
            + current.getRemainingCoins().size();

            if (board.getTileAt(current.getX(), current.getY()).isGoal()
            && current.getRemainingCoins().isEmpty()) {
```

```

        return reconstructPath(current);
    }

    if (visited.contains(stateId)) continue;
    visited.add(stateId);
    for (Direction dir : Direction.values()) {
        Node nextNode = current.move(dir, board);
        if (nextNode != null &&
!visited.contains(nextNode.getStateId())) {
            long h = calculateHeuristic(nextNode.getX(),
nextNode.getY(), nextNode.getRemainingCoins(), board);
            nextNode.setHCost(h);
            pq.add(nextNode);
        }
    }
}
return null;
}
}

```

Langkah-langkah GBFS

1. inisialisasi:
 Nilai heuristik awal dihitung melalui `calculateHeuristic()` berdasarkan posisi awal dan daftar koin. Simpul awal dibentuk dan dimasukkan ke dalam *priority queue* dengan nilai $h(n)$ tersebut.
2. Iterasi utama:
 Selama *priority queue* tidak kosong, algoritma mengambil simpul dengan nilai $h(n)$ terkecil dari antrian melalui operasi `pq.poll()`.
3. Pemeriksaan tujuan:
 Apabila simpul yang sedang diproses berada pada ubin tujuan dan seluruh koin telah dikumpulkan, algoritma berhenti dan jalur direkonstruksi.
4. Pemeriksaan duplikasi:
 Apabila `stateId` dari simpul saat ini telah terdapat dalam `visited`, simpul tersebut dilewati. Jika belum, simpul dicatat ke dalam `visited`.
5. Ekspansi tetangga:
 Seluruh arah pergerakan dieksplorasi. Untuk setiap simpul tetangga yang valid dan belum terdapat dalam `visited`, nilai $h(n)$ dihitung ulang melalui

calculateHeuristic(), kemudian simpul dimasukkan ke dalam *priority queue*.

6. Rekonstruksi jalur:

Setelah simpul tujuan ditemukan, jalur direkonstruksi dengan menelusuri kembali jejak *parent* dari simpul tujuan hingga simpul awal.

7. Kondisi gagal:

Apabila *priority queue* telah kosong dan simpul tujuan belum ditemukan, metode solve() mengembalikan nilai null.

Untuk implementasi GBFS ini tidak akan menjamin solusi optimal dan pencarian tidak selalu mencari semua langkah karena tidak menelusuri seluruh search tree.

2.7.4. Uniform cost search (UCS)

Uniform cost search (UCS) merupakan algoritma pencarian yang mengekskansi simpul berdasarkan biaya aktual terkecil menuju target dengan menggunakan fungsi perhitungan $g(n)$. Sepenuhnya berfokus pada efisiensi meminimalisasi biaya aktual.

Pada implementasi ini digunakan priority queue yang mengutamakan gCost terkecil ke yang terbesar digunakan juga map bernama visited yang berisi nodeid terhadap gcost untuk melakukan early pruning dari sebuah search tree jika nama nodenya sama dan nilai biaya aktual dari node yang sedang diproses lebih besar dibanding perhitungan sebelumnya.

```
public class UCSSolver extends Solver {
    public UCSSolver(Board board) {
        super(board);
    }

    @Override
    public List<Direction> solve() {
        PriorityQueue<Node> pq = new
PriorityQueue<>(Comparator.comparingLong(Node::getGCost));
        Map<String, Long> visited = new HashMap<>(); // nodeId : Gcost
        int startX = board.getPlayer().getXCoords();
        int startY = board.getPlayer().getYCoords();
        Queue<Integer> initialCoins = new
LinkedList<>(board.getCoinOrder());
```



```

        Node StartNode = new Node(startX, startY, initialCoins, 0,0,
null, null);
        pq.add(StartNode);

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            String stateId = current.getStateId();
            if (board.getTileAt(current.getX(), current.getY()).isGoal()
&& current.getRemainingCoins().isEmpty()) {
                return reconstructPath(current);
            }
            if (visited.containsKey(stateId) && visited.get(stateId) <=
current.getGCost()) {
                continue;
            }
            visited.put(stateId, current.getGCost());
            for (Direction dir : Direction.values()) {
                Node nextNode = current.move(dir, board);
                if (nextNode != null) {
                    pq.add(nextNode);
                }
            }
        }
        return null;
    }
}

```

Langkah-langkah UCS:

1. Inisialisasi:
Simpul awal dibentuk dengan posisi pemain pada koordinat awal, daftar koin yang belum diambil, serta nilai $g(n) = 0$. Simpul tersebut dimasukkan ke dalam *priority queue*.
2. Iterasi utama:
Selama *priority queue* tidak kosong, algoritma mengambil simpul dengan nilai $g(n)$ terkecil dari antrian melalui operasi `pq.poll()`.
3. Pemeriksaan tujuan:
Apabila simpul yang sedang diproses berada pada ubin tujuan (`isGoal()`) dan

seluruh koin telah dikumpulkan (`getRemainingCoins().isEmpty()`), algoritma berhenti dan jalur direkonstruksi melalui `reconstructPath(current)`.

4. Pemeriksaan duplikasi:

Apabila `stateId` dari simpul saat ini telah terdapat dalam `visited` dengan nilai $g(n)$ yang lebih kecil atau sama, simpul tersebut dilewati (*pruning*). Jika belum, simpul dicatat ke dalam `visited` beserta nilai $g(n)$ -nya.

5. Ekspansi tetangga:

Seluruh arah pergerakan yang tersedia (`Direction.values()`) dieksplorasi. Untuk setiap arah, dihasilkan simpul baru melalui `current.move(dir, board)`. Apabila pergerakan valid (tidak null), simpul baru dimasukkan ke dalam *priority queue* dengan nilai $g(n)$ yang telah diperbarui.

6. Rekonstruksi jalur:

Setelah simpul tujuan ditemukan, jalur optimal direkonstruksi dengan menelusuri kembali jejak *parent* dari simpul tujuan hingga simpul awal.

7. Kondisi gagal:

Apabila *priority queue* telah kosong dan simpul tujuan belum ditemukan, metode `solve()` mengembalikan nilai null.

Pada algoritma ini akan terjamin optimal selama biaya langkah tidak ada yang bernilai negatif dan UCS ini akan memastikan menemukan solusi jika solusi ada. Secara efisiensi algoritma ini rendah karena harus menyusuri ruang pencarian untuk mendapatkan solusi optimal tanpa bantuan heuristik.

2.7.5. *A* Search*

A Search* adalah salah satu algoritma pencarian jalur yang paling populer dan efisien karena menggabungkan informasi biaya aktual serta estimasi heuristik untuk mencapai tujuan. Algoritma ini bersifat optimal (menjamin jalur terpendek) dan komplit (pasti menemukan jalur jika ada), selama fungsi heuristik yang digunakan bersifat *admissible* (tidak pernah melebihi-lebihkan biaya asli). Dengan fungsi evaluasi utama sebagai berikut *A** bekerja dengan meminimalkan fungsi total biaya $f(n)$, yang dirumuskan sebagai:

$$f(n) = g(n) + h(n)$$

$g(n)$ (Cost): Biaya aktual yang telah ditempuh dari titik awal ke node n saat ini. Ini memastikan algoritma tetap realistis terhadap jarak yang sudah dilalui.

$h(n)$ (Heuristic): Estimasi biaya dari node n ke titik tujuan. Ini memberikan "insting" kepada algoritma untuk bergerak ke arah target.

Pada implementasi kami, A* implementasinya seperti dengan UCS namun digunakan tambahan dengan perhitungan heuristik pada pengurutan priority queue-nya. A* kami menggunakan jarak Manhattan sebagai biaya heuristiknya yang *admissible* sehingga algoritma ini dijamin bisa mendapatkan jalur optimal.

```
public class AStarSolver extends Solver{
    public AStarSolver(Board board) {
        super(board);
    }

    @Override
    public List<Direction> solve() {
        PriorityQueue<Node> pq = new
PriorityQueue<>(Comparator.comparingLong(n -> n.getGCost() +
n.getHCost()));
        Map<String, Long> visited = new HashMap<>();
        int startX = board.getPlayer().getXCoords();
        int startY = board.getPlayer().getYCoords();
        Queue<Integer> initialCoins = new
LinkedList<>(board.getCoinOrder());
        Node StartNode = new Node(startX, startY, initialCoins, 0, 0,
null, null);
        pq.add(StartNode);

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            String stateId = current.getStateId();
            if (board.getTileAt(current.getX(), current.getY()).isGoal()
&& current.getRemainingCoins().isEmpty()) {
                return reconstructPath(current);
            }

            if (visited.containsKey(stateId) && visited.get(stateId) <=
current.getGCost()) {
                continue;
            }
            visited.put(stateId, current.getGCost());

            for (Direction dir : Direction.values()) {
                Node nextNode = current.move(dir, board);
                if (nextNode != null) {
```

```

        long newHCost = calculateHeuristic(nextNode.getX(),
nextNode.getY(), nextNode.getRemainingCoins(),
        board);
        nextNode.setHCost(newHCost);
        pq.add(nextNode);
    }
}

}
return null;
}
}

```

Langkah-langkah A*

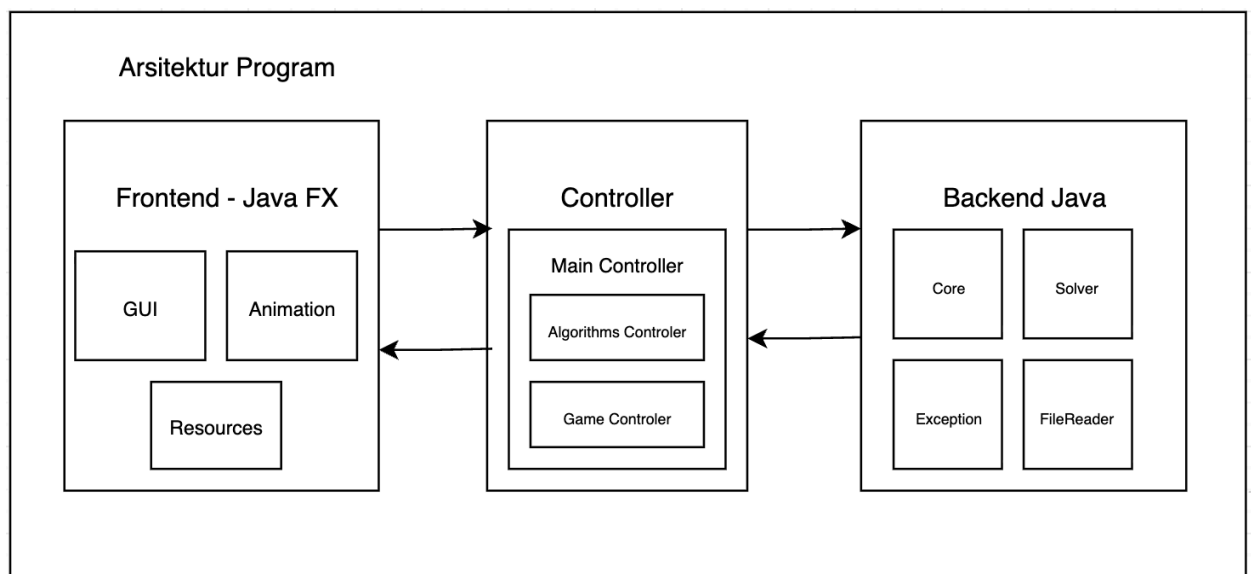
1. Inisialisasi: Simpul awal dibentuk dengan nilai $g(n) = 0$ dan $h(n) = 0$. Simpul tersebut dimasukkan ke dalam *priority queue*. Nilai $h(n)$ akan dihitung secara dinamis pada saat ekspansi tetangga.
2. Iterasi utama: Selama *priority queue* tidak kosong, algoritma mengambil simpul dengan nilai $f(n) = g(n) + h(n)$ terkecil dari antrian melalui operasi `pq.poll()`.
3. Pemeriksaan tujuan: Apabila simpul yang sedang diproses berada pada ubin tujuan dan seluruh koin telah dikumpulkan, algoritma berhenti dan jalur direkonstruksi melalui `reconstructPath(current)`.
4. Pemeriksaan duplikasi: Apabila `stateId` dari simpul saat ini telah terdapat dalam `visited` dengan nilai $g(n)$ yang lebih kecil atau sama, simpul tersebut dilewati (*pruning*). Jika belum, simpul dicatat ke dalam `visited` beserta nilai $g(n)$ -nya.
5. Ekspansi tetangga: Seluruh arah pergerakan dieksplorasi. Untuk setiap simpul tetangga yang valid, nilai $h(n)$ dihitung melalui `calculateHeuristic()` dan ditetapkan melalui `setHCost()`. Simpul tetangga kemudian dimasukkan ke dalam *priority queue* dengan nilai $f(n)$ yang telah diperbarui.
6. Rekonstruksi jalur: Setelah simpul tujuan ditemukan, jalur optimal direkonstruksi dengan menelusuri kembali jejak *parent* dari simpul tujuan hingga simpul awal.
7. Kondisi gagal: Apabila *priority queue* telah kosong dan simpul tujuan belum ditemukan, metode `solve()` mengembalikan nilai `null`.

Algoritma A* ini akan menjamin solusi optimal jika heuristik yang dipakai *admissible* dan solusi yang didapatkan akan menjamin solusi karena menelusuri semua ruang pencarian dan lebih efektif karena menggunakan heuristik.

3. Arsitektur Program

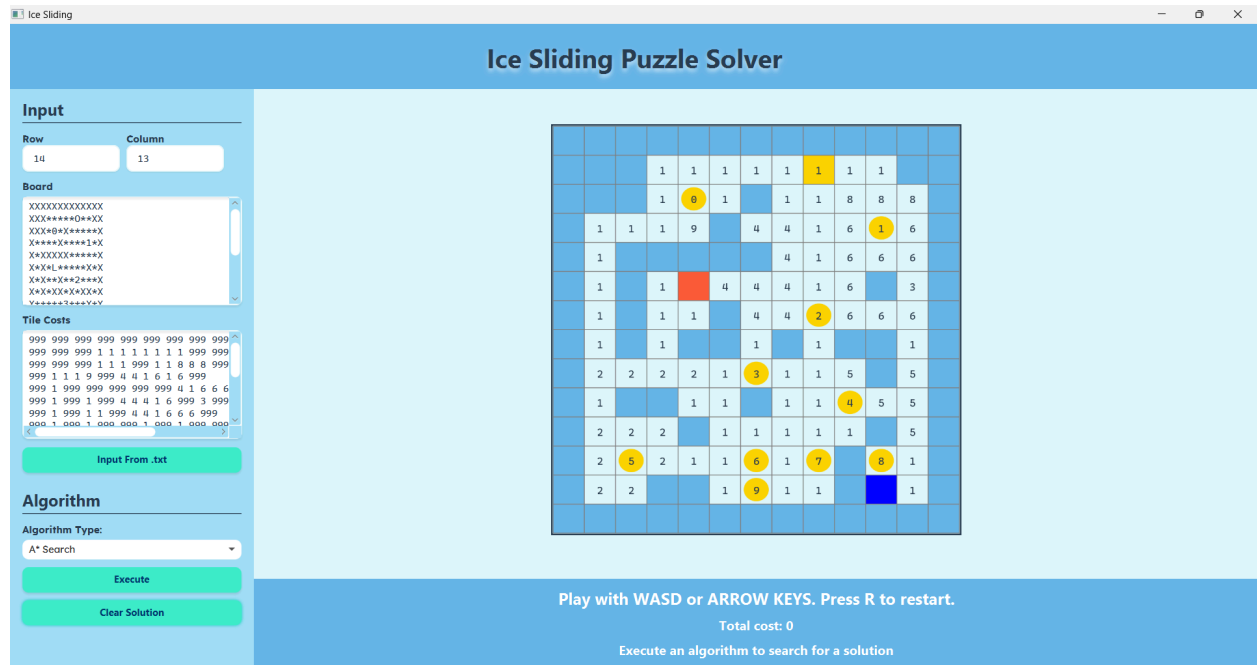
3.1. Struktur Perangkat Lunak

Secara umum Arsitektur kami membagi menjadi 3 bagian mengikuti framework MVC, yaitu Model, View dan Controller. Frontend di sini untuk menampilkan permainan untuk user seperti FXML tampilan animasi dan GUI untuk jenis-jenis objek lainnya. Controller berfungsi untuk mengatur objek-objek frontend dan juga sebagai perantara komunikasi terhadap backend. Terakhir backend adalah tempat pencarian rute, membaca file dan logika utama bagaimana permainan ini bekerja.

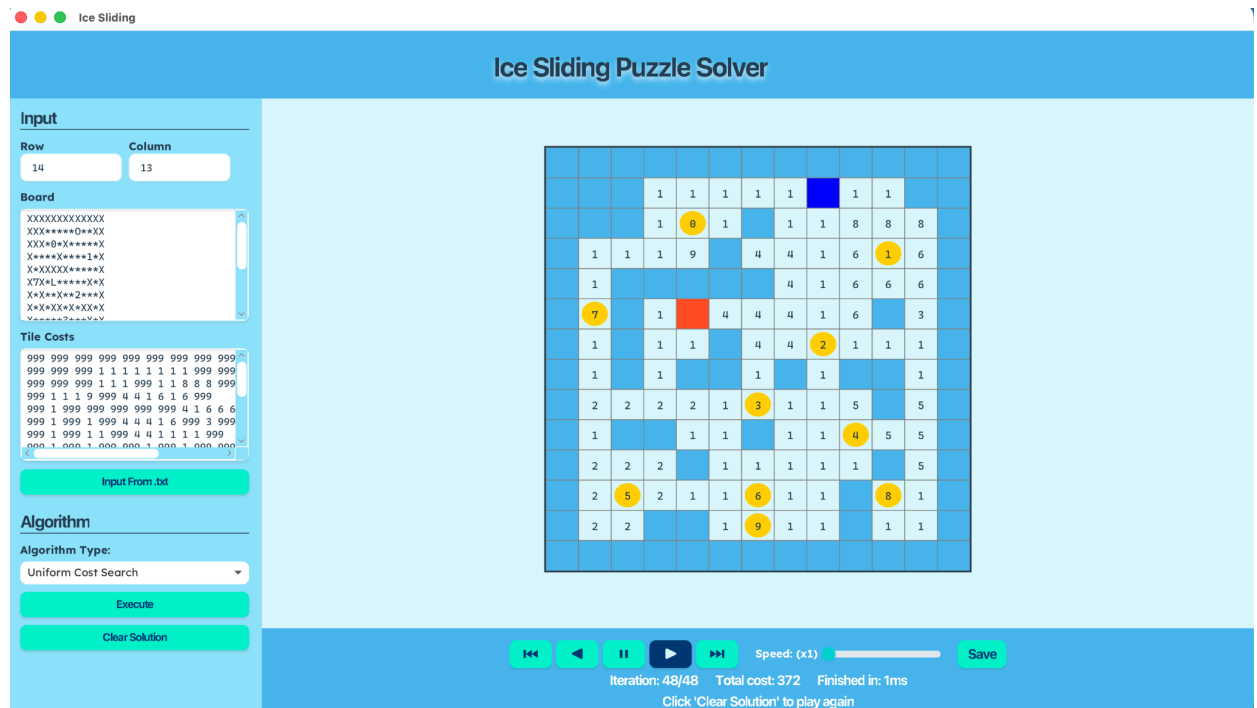


3.2. Bonus GUI

GUI diimplementasikan dalam Java menggunakan JavaFX. Tampilan menggunakan palet warna yang bertemakan es karena namanya juga *Ice Sliding Puzzle Solver*.



Tampilan utama aplikasi



Tampilan aplikasi saat algoritma dijalankan

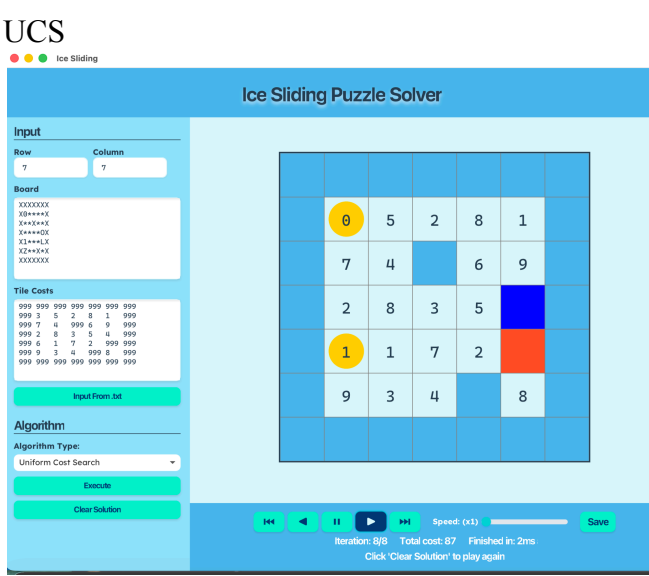
Dalam aplikasi, player direpresentasikan dengan kotak berwarna biru tua. Petak yang bisa dilangkahi berwarna biru muda dengan angka berupa "cost"-nya. Petak dinding berwarna biru biasa. Petak angka atau koin berupa lingkaran kuning dengan angka urutannya. Petak lava berupa kotak berwarna oren. Terakhir petak finish berwarna kuning.

Pada bagian kiri aplikasi, terdapat bagian input. Papan akan berubah langsung sesuai dengan input yang diberikan jika input valid. Input dengan file juga dapat dilakukan. Algoritma dapat dipilih dan dieksekusi dari bagian ini juga.

Di bagian tengah/kanan, ada papan yang dapat dimainkan jika belum ada algoritma yang dieksekusi. Pengguna dapat bermain dengan menekan 'W', 'A', 'S', 'D' dan arrow key untuk bergerak serta tombol 'R' untuk restart. Total cost yang didapat juga bertambah setiap kali melangkah. Terdapat pesan juga jika player menang atau kalah.

Di bagian bawah, terdapat juga implementasi animasi untuk menunjukkan animasi perlangkah ditambah dengan tombol playback maju mundur dan kebebasan user untuk mengatur kecepatan. Tombol clear solution dapat ditekan supaya permainan dapat dimainkan lagi. Fitur akhir kami adalah menyediakan fitur save pada eksekusi program untuk menunjukkan langkah dan total cost dari algoritma yang dipilih dalam bentuk text file (.txt).

4. Pengujian

No	Test Case	Hasil
1	<pre> 77 XXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999 </pre>	 UCS GBFS

● ● ● Ice Sliding

Ice Sliding Puzzle Solver

Input

Row: 18 Column: 18

Board

Tile Costs

Algorithm

Uniform Cost Search

Execute

Clear Solution

1	2	3	4	5	6	7		8	9
1		2	3	4		0	0	7	8
1	0	0		2	5	0	0	8	9
1	0	0	0	0	0	0	0		0
1	0		3	4	5	0	7	8	9
1	0	3	4	5	6	0		8	9
1	0	3	4		5	0	7	8	9
1	2	0	0	0	1	0	0		9
1	2	3	4	5	6	0			0
1	2	3	4	5	6			9	0

Iteration: 8/8 Total cost: 0 Finished in: 2ms
Click 'Clear Solution' to play again

● ● ● Ice Sliding

Ice Sliding Puzzle Solver

Input

Row: 18 Column: 18

Board

Tile Costs

Algorithm

Greedy Best First Search

Execute

Clear Solution

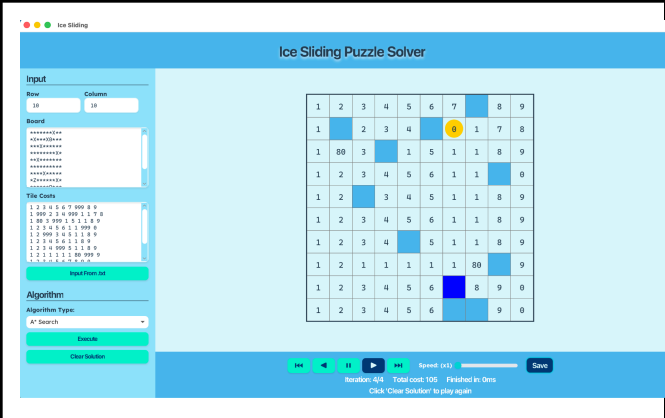
1	2	3	4	5	6	7		8	9
1		2	3	4		0	0	7	8
1	0	0		2	5	0	0	8	9
1	0	0	0	0	0	0	0		0
1	0		3	4	5	0	7	8	9
1	0	3	4	5	6	0		8	9
1	0	3	4		5	0	7	8	9
1	2	0	0	0	1	0	0		9
1	2	3	4	5	6	0			0
1	2	3	4	5	6			9	0

Iteration: 2/2 Total cost: 1 Finished in: 3ms
Click 'Clear Solution' to play again

AStar Search

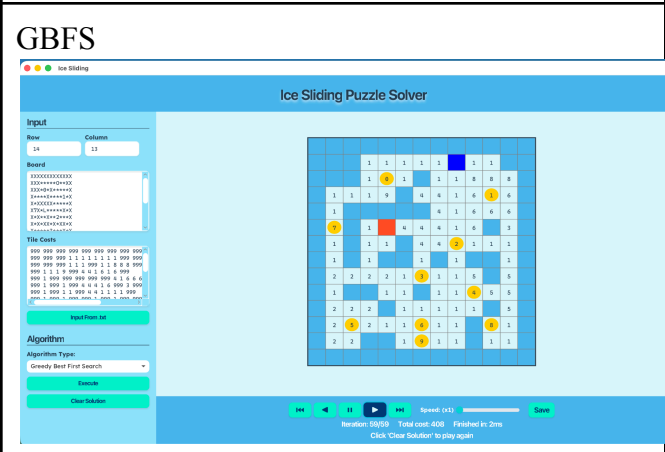
		Ice Sliding Puzzle Solver Input: Row 18, Column 18 Board: [Visual representation of the 9x9 grid] Tile Costs: [List of costs for each tile] Algorithm: A* Search Buttons: Execute, Clear Solution Iteration: 2/2, Total cost: 1, Finished in: 1ms
3	<pre> 10 10 **X*X*X** *XO**X**** ***X***** *****X* *1X***** X***0**X** ***X***** *Z*****X* ***** *****X** 12 999 4 5 6 7 999 8 9 1 999 2 999 4 999 5 6 7 8 12 3 4 999 5 6 7 8 9 12 3 4 5 6 7 8 999 0 12 999 3 4 5 6 7 8 9 1 999 3 4 5 6 7 999 8 9 12 3 4 999 5 6 7 8 9 12 3 4 5 6 7 8 999 9 12 3 4 5 6 7 8 9 0 12 3 4 5 6 7 999 9 0 </pre>	UCS Ice Sliding Puzzle Solver Input: Row 14, Column 13 Board: [Visual representation of the 14x13 grid] Tile Costs: [List of costs for each tile] Algorithm: Uniform Cost Search Buttons: Execute, Clear Solution Iteration: 48/48, Total cost: 372, Finished in: 5ms
		GBFS Ice Sliding Puzzle Solver Input: Row 18, Column 18 Board: [Visual representation of the 18x18 grid] Tile Costs: [List of costs for each tile] Algorithm: Greedy Best First Search Buttons: Execute, Clear Solution Iteration: 10/10, Total cost: 2103, Finished in: 0ms
		A*

4	<pre> 10 10 *****X** *X***X0** ***X***** *****X* **X***** ***** ****X***** *Z*****X* *****O*** *****XX** 1 2 3 4 5 6 7 999 8 9 1 999 2 3 4 999 1 1 7 8 1 80 3 999 1 5 1 1 8 9 1 2 3 4 5 6 1 1 999 0 1 2 999 3 4 5 1 1 8 9 1 2 3 4 5 6 1 1 8 9 1 2 3 4 999 5 1 1 8 9 1 2 1 1 1 1 1 1 80 999 9 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 999 999 9 0 </pre>	UCS
		GBFS
		AStar



5

14 13
XXXXXXXXXXXXXXXXX
X***X*O***X
X*X*0*X*X***X
X**X***X*1*X
X*XXXXX*X***X
X*X*L***X*XXX
X*X*X**2***X
X*X*XXXXXXXXXX
X***X*3*X***X
XLXX**X*4**X
X***X*****X*X
X*5*X*6*7*8*X
X**XX*9*X*Z*X
XXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999 999
999 1 1 1 1 1 999 1 1 1 1 1 999
999 1 999 1 1 1 999 1 999 8 8 8 999
999 1 1 1 999 4 4 4 999 6 1 6 999
999 1 999 999 999 999 999 4 999 6 6 6 999
999 1 999 1 999 4 4 4 999 6 999 999 999
999 1 999 1 1 999 4 4 1 6 6 6 999
999 1 999 1 999 999 999 999 999 999 999 999 999
999 2 2 2 999 1 1 1 999 5 5 5 999
999 999 999 999 1 1 999 1 1 1 5 5 999
999 2 2 2 999 1 1 1 1 1 999 5 999
999 1 1 2 999 1 1 1 1 1 1 1 999
999 2 2 999 999 1 1 1 999 1 1 1 999
999 999 999 999 999 999 999 999 999 999 999 999 999



A*

The screenshot displays the 'Ice Sliding Puzzle Solver' application. The 'Input' section shows a 14x13 grid. The board visualization shows a 14x13 grid with numbered tiles (1-13) and a blue goal area. The solver has found a solution in 296 moves, with a total cost of 372 and a finish time of 29ms. The bottom status bar shows navigation buttons (Back, Home, Forward, Stop) and a 'Save' button.

[illegible][illegible][illegible] A^*

[illegible]

5. Kesimpulan dan Saran

Algoritma pencarian jalur pada graf secara heuristik dapat diterapkan untuk mencari solusi dari berbagai permasalahan, contohnya dalam permainan *Ice Sliding Puzzle*. Algoritma berhasil mencari solusi dalam waktu yang cepat karena tidak semua *node* kondisi ditelusuri.

Dari pengujian yang dilakukan, algoritma A* Search bisa mendapatkan jumlah langkah dan cost yang lebih atau sama optimal dengan algoritma Uniform Cost Search dan Greedy Best First Search karena A* Search dapat dikatakan sebagai gabungan dua algoritma tersebut. Oleh karena itu, algoritma A* Search direkomendasikan untuk pencarian solusi secara heuristik, utamanya untuk path finding.

Aplikasi yang telah dibuat cukup memuaskan bagi kami karena permainan dapat dimainkan dan juga dicari solusinya. Ditambah dengan animasi yang cukup satisfying karena animasi gerakan player. Adapun hal-hal yang masih kurang atau dapat ditingkatkan adalah sebagai berikut:

- Animasi masih bisa ngebug jika mencoba "clear solution" saat animasi berjalan.
- Hasil "No solution :(" mungkin kurang terasa beda dibandingkan saat berhasil menemukan solusi.
- Perbagus pesan menang atau kalah dengan mengganti warna tulisan.
- Tambahan animasi koin terambil dan pemain keluar papan atau mati.
- Tambahan petunjuk aturan permainan dan input yang valid.
- Tambahan algoritma dan fungsi heuristik yang dapat dipilih.

Lampiran

Repositori github: https://github.com/HussainDzaki/MA3052_Tugas_13524018_13524084

Hasil Test case :  HasilTestCaseTucil3

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



[Muhammad Nafis Habibi]



[Dzaki Ahmad Al Hussainy]

Laporan Pengerjaan

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)		✓

Daftar Pustaka

[Uniform Cost Search \(UCS\) in AI - GeeksforGeeks](#)

▶ Uniform Cost Search Algorithm | UCS Search Algorithm in Artificial Intelligence by Mah...

[Greedy Best first search algorithm - GeeksforGeeks](#)

[A* Search Algorithm - GeeksforGeeks](#)

▶ A* Pathfinding (E01: algorithm explanation)

▶ A* Pathfinding (E03: algorithm implementation)